

Laporan Tugas Kecil 3

IF2211 Strategi Algoritma

Ice Sliding Puzzle Solver
UCS, Greedy Best-First Search, dan A*

Nama : Miguel Rangga Deardo Sinaga
NIM : 13524069

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1	Pendahuluan	3
1.1	Deskripsi Masalah	3
1.2	Elemen Papan	3
1.3	Aturan Sliding	3
2	Algoritma <i>Pathfinding</i> yang Digunakan	4
2.1	Representasi State	4
2.2	Uniform Cost Search (UCS)	4
2.2.1	Deskripsi	4
2.2.2	Langkah-Langkah	4
2.2.3	Definisi Fungsi	5
2.3	Greedy Best-First Search (GBFS)	5
2.3.1	Deskripsi	5
2.3.2	Langkah-Langkah	5
2.3.3	Definisi Fungsi	6
2.4	A* Search	6
2.4.1	Deskripsi	6
2.4.2	Langkah-Langkah	6
2.4.3	Definisi Fungsi	6
3	Fungsi Heuristik	7
3.1	H1 — Manhattan Distance	7
3.2	H2 — Chebyshev Distance	7
3.3	H3 — Minimum Moves	7
4	Analisis Algoritma	8
4.1	Admissibilitas Heuristik pada A*	8
4.2	Perbandingan Teoritis Algoritma	9
4.2.1	Analisis pada Konteks Ice Sliding	9
5	Implementasi Program	9
5.1	Bahasa dan Struktur Program	9
5.2	Format File Input	10
5.3	Cara Menjalankan Program	10
5.4	Cuplikan Kode Utama	11
5.4.1	Simulasi Sliding (Board.java)	11
5.4.2	Implementasi A* (AStar.java)	12

6	Hasil Percobaan	13
6.1	Kasus Uji	13
6.1.1	Kasus Uji 1	14
6.1.2	Kasus Uji 2	15
6.1.3	Kasus Uji 3	15
6.1.4	Kasus Uji 4	16
6.1.5	Kasus Uji 5	17
6.2	Rekap Data Percobaan	18
6.3	Analisis Hasil Percobaan	18
6.3.1	Optimalitas Solusi	18
6.3.2	Pengaruh Ukuran Papan dan Checkpoint	19
6.3.3	Pengaruh Lava sebagai Rintangan	19
6.3.4	Perbandingan Heuristik	19
7	Bonus: Antarmuka Grafis (GUI)	19
8	Pranala Repository	20
9	Kesimpulan	20

1 Pendahuluan

1.1 Deskripsi Masalah

Ice Sliding Puzzle adalah sebuah permainan teka-teki di atas sebuah papan berukuran $N \times M$. Seorang aktor ditempatkan pada posisi awal (Z) dan harus mencapai posisi tujuan (O) dengan mematuhi aturan fisika es: ketika aktor bergerak ke suatu arah, ia akan **terus meluncur** hingga menabrak dinding (X) atau sisi papan.

Setiap perpindahan memiliki biaya (cost) sesuai nilai yang tertera pada setiap petak yang dilalui. Tujuannya adalah menemukan jalur dari Z ke O (melalui semua *checkpoint* secara berurutan, jika ada) dengan meminimalkan total biaya perjalanan.

1.2 Elemen Papan

Simbol	Nama	Keterangan
*	Path	Petak kosong yang dapat dilalui
X	Dinding	Menghalangi pergerakan; aktor berhenti <i>sebelum</i> petak ini
L	Lava	Mematikan; jika dilalui, gerakan tidak valid
Z	Start	Posisi awal aktor
O	Goal	Posisi tujuan; aktor berhenti di sini <i>hanya jika</i> semua <i>checkpoint</i> selesai
0-9	Checkpoint	Harus dikunjungi secara berurutan sebelum mencapai tujuan

Tabel 1: Elemen-elemen pada papan *Ice Sliding Puzzle*

1.3 Aturan Sliding

1. Aktor bergerak ke salah satu dari empat arah: Atas (U), Bawah (D), Kiri (L), Kanan (R).
2. Aktor terus meluncur pada arah tersebut hingga petak berikutnya adalah dinding (X) atau sisi papan.
3. Jika lintasan melewati petak L (lava), gerakan tersebut tidak valid.
4. Jika lintasan melewati *checkpoint* k sementara *checkpoint* berikutnya yang diharapkan bukan k , gerakan tidak valid.
5. Petak O tidak menghentikan aktor kecuali seluruh *checkpoint* telah dikunjungi.

2 Algoritma *Pathfinding* yang Digunakan

2.1 Representasi State

Sebuah *state* dalam ruang pencarian didefinisikan sebagai pasangan:

$$s = (\text{row}, \text{col}, \text{nextCheckpoint})$$

di mana *nextCheckpoint* adalah indeks *checkpoint* berikutnya yang harus dikunjungi (-1 jika semua sudah terpenuhi). Dua posisi yang sama tetapi berbeda status *checkpoint* merupakan *state* yang berbeda.

State awal: $s_0 = (r_Z, c_Z, 0)$ (atau -1 jika tidak ada checkpoint).

State tujuan: $s^* = (r_O, c_O, -1)$.

2.2 Uniform Cost Search (UCS)

2.2.1 Deskripsi

UCS adalah algoritma pencarian *uninformed* yang memperluas *node* dengan biaya aktual terkecil terlebih dahulu. UCS merupakan versi umum dari algoritma Dijkstra pada graf dengan bobot sisi positif.

2.2.2 Langkah-Langkah

Langkah 1. Inisialisasi. Buat *state* awal s_0 dengan $g(s_0) = 0$. Masukkan s_0 ke dalam *priority queue* $\mathcal{Q}$ (min-heap berdasarkan g). Buat peta **bestG** untuk menyimpan nilai g terbaik yang pernah ditemukan per *state*.

Langkah 2. Iterasi. Selama $\mathcal{Q}$ tidak kosong, ambil *state* s dengan $g(s)$ terkecil dari $\mathcal{Q}$.

Langkah 3. Lazy deletion. Jika $g(s) > \text{bestG}[s]$, lewati *state* ini (sudah digantikan oleh jalur yang lebih murah).

Langkah 4. Cek tujuan. Jika $s = s^*$, hentikan pencarian dan rekonstruksi jalur dengan mengikuti pointer **parent**.

Langkah 5. Ekspansi. Untuk setiap arah $d \in \{U, D, L, R\}$, simulasikan gerakan *sliding* dari s ke arah d :

- (a) Jika gerakan tidak valid (lava/keluar papan/checkpoint salah urutan), lewati.
- (b) Hitung $g(s') = g(s) + \text{cost_gerakan}$.
- (c) Jika $g(s') < \text{bestG}[s']$, perbarui **bestG** $[s']$ dan masukkan s' ke $\mathcal{Q}$.

Langkah 6. Tidak ada solusi. Jika $\mathcal{Q}$ kosong sebelum tujuan ditemukan, nyatakan tidak ada solusi.

2.2.3 Definisi Fungsi

Fungsi UCS

$$f(n) = g(n)$$

$$g(n) = \text{total biaya aktual dari } s_0 \text{ ke } n$$

$$h(n) = 0 \quad (\text{tidak digunakan})$$

2.3 Greedy Best-First Search (GBFS)

2.3.1 Deskripsi

GBFS adalah algoritma pencarian *informed* yang sepenuhnya mengandalkan fungsi heuristik $h(n)$ untuk memandu pencarian. Algoritma ini selalu memperluas *node* dengan estimasi jarak terkecil ke tujuan, tanpa mempertimbangkan biaya yang sudah dikeluarkan.

2.3.2 Langkah-Langkah

Langkah 1. Inisialisasi. Hitung $h(s_0)$ menggunakan fungsi heuristik yang dipilih. Masukkan s_0 ke *priority queue* $\mathcal{Q}$ (min-heap berdasarkan h). Buat himpunan *visited* yang kosong.

Langkah 2. Iterasi. Selama $\mathcal{Q}$ tidak kosong, ambil *state* s dengan $h(s)$ terkecil.

Langkah 3. Cek visited. Jika $s \in \text{visited}$, lewati. Tambahkan s ke *visited*.

Langkah 4. Cek tujuan. Jika $s = s^*$, hentikan dan rekonstruksi jalur.

Langkah 5. Ekspansi. Untuk setiap arah d , simulasikan *sliding*:

- (a) Jika tidak valid, lewati.
- (b) Hitung $h(s')$ untuk *state* baru s' .
- (c) Jika $s' \notin \text{visited}$, masukkan s' ke $\mathcal{Q}$ dengan prioritas $h(s')$.

Langkah 6. Tidak ada solusi. Jika $\mathcal{Q}$ kosong, nyatakan tidak ada solusi.

2.3.3 Definisi Fungsi

Fungsi GBFS

$$f(n) = h(n)$$

$g(n)$ = tidak digunakan untuk menentukan prioritas

$h(n)$ = estimasi heuristik jarak dari n ke tujuan

2.4 A* Search

2.4.1 Deskripsi

A* menggabungkan kekuatan UCS (menjamin optimalitas melalui g) dan GBFS (efisiensi melalui h). A* memperluas *node* dengan nilai $f(n) = g(n) + h(n)$ terkecil. Jika heuristik *admissible*, A* menjamin solusi optimal.

2.4.2 Langkah-Langkah

Langkah 1. Inisialisasi. Hitung $h(s_0)$ dan $f(s_0) = 0 + h(s_0)$. Masukkan s_0 ke *priority queue* Q (min-heap berdasarkan f). Buat peta **bestG**.

Langkah 2. Iterasi. Ambil *state* s dengan $f(s)$ terkecil.

Langkah 3. Lazy deletion. Lewati jika $g(s) > \text{bestG}[s]$.

Langkah 4. Cek tujuan. Jika $s = s^*$, selesai.

Langkah 5. Ekspansi. Untuk setiap arah d :

- (a) Jika tidak valid, lewati.
- (b) $g(s') = g(s) + \text{cost_gerakan}$.
- (c) $h(s') = \text{heuristik}(s')$.
- (d) $f(s') = g(s') + h(s')$.
- (e) Jika $g(s') < \text{bestG}[s']$, perbarui dan masukkan ke Q .

2.4.3 Definisi Fungsi

Fungsi A*

$$f(n) = g(n) + h(n)$$

$g(n)$ = biaya aktual dari s_0 ke n

$h(n)$ = estimasi heuristik dari n ke tujuan (*admissible*)

3 Fungsi Heuristik

Pada setiap *state* n , **target sementara** ditentukan sebagai berikut: jika masih ada *checkpoint* yang harus dikunjungi, target adalah posisi *checkpoint* berikutnya; jika tidak, target adalah posisi tujuan O .

Misalkan $\Delta r = |\text{row}(n) - \text{row}(\text{target})|$ dan $\Delta c = |\text{col}(n) - \text{col}(\text{target})|$.

3.1 H1 — Manhattan Distance

$$h_1(n) = \Delta r + \Delta c$$

Alasan admissible: Pada *ice sliding*, setiap petak yang dilalui memiliki biaya ≥ 1 . Jarak Manhattan adalah batas bawah jumlah petak minimum yang harus dilalui antara dua titik pada grid. Oleh karena itu, $h_1(n) \leq h^*(n)$.

3.2 H2 — Chebyshev Distance

$$h_2(n) = \max(\Delta r, \Delta c)$$

Alasan admissible: Chebyshev Distance $\leq$ Manhattan Distance $\leq h^*(n)$. Satu gerakan *sliding* menempuh setidaknya $\max(\Delta r, \Delta c)$ petak ke arah yang tepat, sehingga ini merupakan *lower bound* yang valid. H2 cenderung memberikan estimasi yang lebih kecil dari H1, sehingga lebih *optimistic* dan tetap admissible.

3.3 H3 — Minimum Moves

$$h_3(n) = \begin{cases} 0 & \text{jika } n = \text{target} \\ 1 & \text{jika segaris horizontal atau vertikal dengan target} \\ 2 & \text{selainnya} \end{cases}$$

Alasan admissible: Nilai 0, 1, dan 2 merepresentasikan jumlah gerakan minimum yang diperlukan secara geometris. Karena $h^*(n) \geq h_3(n)$ secara geometris, heuristik ini admissible. Namun, H3 kurang informatif dibanding H1 dan H2 karena bukan fungsi dari biaya petak.

4 Analisis Algoritma

4.1 Admissibilitas Heuristik pada A*

Definisi Admissible

Suatu heuristik $h(n)$ dikatakan **admissible** jika untuk setiap *node* n :

$$h(n) \leq h^*(n)$$

di mana $h^*(n)$ adalah biaya aktual optimal dari n ke tujuan.

Ketiga heuristik yang diimplementasikan (H1, H2, H3) bersifat admissible dengan asumsi bahwa setiap petak memiliki biaya ≥ 1 . Pembuktiannya:

1. **H1 (Manhattan):** Jalur optimal dari n ke target harus menempuh setidaknya Δr perpindahan baris dan Δc perpindahan kolom. Dengan biaya petak ≥ 1 , maka cost optimal $\geq \Delta r + \Delta c = h_1(n)$.
2. **H2 (Chebyshev):** Karena $h_2 \leq h_1$ dan H1 admissible, maka H2 juga admissible.
3. **H3 (Min Moves):** Nilai maksimum H3 adalah 2 (dua gerakan). Pada hampir semua kasus non-trivial, $h^*(n) \geq 1$, sehingga $h_3 \leq h^*$.

Dengan demikian, A* menggunakan H1, H2, atau H3 pada permasalahan ini **dijamin menghasilkan solusi optimal** (biaya minimum) jika ada.

4.2 Perbandingan Teoritis Algoritma

Kriteria	UCS	GBFS	A*
Completeness	Ya (jika $\text{cost} > 0$)	Ya (pada ruang terbatas)	Ya (jika h admissible)
Optimalitas	Ya	Tidak	Ya (jika h admissible)
Kompleksitas waktu	$O(b^{C^*/\epsilon})$	$O(b^m)$ (worst)	$O(b^d)$ (dengan h baik)
Kompleksitas ruang	$O(b^{C^*/\epsilon})$	$O(b^m)$	$O(b^d)$
Penggunaan $g(n)$	Ya	Tidak	Ya
Penggunaan $h(n)$	Tidak	Ya	Ya
Karakteristik	Eksplorasi merata	Terarah tapi tak optimal	Seimbang, paling efisien

Tabel 2: Perbandingan teoritis UCS, GBFS, dan A* (b = branching factor, d = kedalaman solusi, m = kedalaman maksimum, C^* = biaya solusi optimal, ϵ = biaya minimum satu langkah)

4.2.1 Analisis pada Konteks Ice Sliding

Pada permasalahan *Ice Sliding Puzzle* ini:

- **Branching factor** efektif kecil (≤ 4 arah) karena banyak gerakan tidak valid (lava, batas papan, checkpoint salah urutan).
- **UCS** akan mengeksplorasi banyak *state* dengan biaya rendah terlebih dahulu, yang mungkin tidak menuju tujuan. Ini membuat UCS lambat pada papan besar dengan biaya petak yang serupa.
- **GBFS** dapat sangat cepat jika heuristiknya informatif, tetapi dapat terjebak pada solusi suboptimal atau mengeluarkan banyak iterasi jika heuristiknya menyesatkan.
- **A*** umumnya paling efisien karena menggabungkan informasi biaya aktual dan estimasi heuristik. Dengan H1 (Manhattan Distance), A* biasanya mengeksplorasi jauh lebih sedikit *state* dibanding UCS.

5 Implementasi Program

5.1 Bahasa dan Struktur Program

Program diimplementasikan menggunakan **Java** dan terdiri dari dua modul utama: modul *solver* (CLI) dan modul GUI.

File	Deskripsi
src/Parser.java	Membaca dan memvalidasi file input <code>.txt</code>
src/Board.java	Representasi papan dan simulasi fisika <i>sliding</i>
src/State.java	Node pencarian: posisi + status <i>checkpoint</i>
src/SolveResult.java	Pembungkus hasil pencarian
src/Heuristic.java	Tiga fungsi heuristik (H1, H2, H3)
src/UCS.java	Implementasi Uniform Cost Search
src/GBFS.java	Implementasi Greedy Best-First Search
src/AStar.java	Implementasi A* Search
src/Playback.java	Visualisasi solusi langkah demi langkah (CLI)
src/Main.java	<i>Entry point</i> mode CLI
gui/Theme.java	Sistem warna pastel dan mode gelap/terang
gui/UI.java	Komponen Swing kustom (<i>button</i> , kartu, <i>tag</i>)
gui/BoardPanel.java	Panel visualisasi grid dengan animasi aktor
gui/ControlPanel.java	Panel input dan kontrol pencarian
gui/ResultPanel.java	Panel statistik dan kontrol <i>playback</i>
gui/MainWindow.java	Jendela utama GUI
gui/GUIMain.java	<i>Entry point</i> mode GUI

Tabel 3: Struktur file program

5.2 Format File Input

```

1 N M
2 [baris 1 peta: M karakter]
3 [baris 2 peta: M karakter]
4 ...
5 [baris N peta: M karakter]
6 [baris 1 cost: M integer]
7 [baris 2 cost: M integer]
8 ...
9 [baris N cost: M integer]
```

Listing 1: Format file input `.txt`

5.3 Cara Menjalankan Program

```

1 # Kompilasi (dari root folder proyek)
2 javac src/*.java gui/*.java
3
4 # Jalankan mode GUI
5 java gui.GUIMain
6
7 # Jalankan mode CLI
```

```
8 java src.Main
```

Listing 2: Kompilasi dan eksekusi

5.4 Cuplikan Kode Utama

5.4.1 Simulasi Sliding (Board.java)

```
1 public SlideResult slide(int startRow, int startCol, int
   dir, int nextNeeded) {
2     int dr = DR[dir], dc = DC[dir];
3     int curRow = startRow, curCol = startCol;
4     int totalCost = 0, stepCount = 0;
5     int currentNextNeeded = nextNeeded;
6
7     while (true) {
8         int nextRow = curRow + dr, nextCol = curCol + dc;
9         // Keluar papan -> tidak valid
10        if (nextRow < 0 || nextRow >= N || nextCol < 0 ||
            nextCol >= M)
11            return new SlideResult();
12
13        char nextTile = grid[nextRow][nextCol];
14
15        if (nextTile == 'X') { // dinding: berhenti di
            posisi sekarang
16            if (stepCount == 0) return new SlideResult(); //
                tidak bergerak
17            return new SlideResult(curRow, curCol,
                totalCost, ...);
18        }
19        if (nextTile == 'L') return new SlideResult(); //
            lava: tidak valid
20
21        if (nextTile >= '0' && nextTile <= '9') {
22            int digit = nextTile - '0';
23            // Checkpoint salah urutan -> tidak valid
24            if (currentNextNeeded != -1 && digit !=
                currentNextNeeded)
25                return new SlideResult();
26            if (digit == currentNextNeeded) {
27                currentNextNeeded++;
```

```

28         if (currentNextNeeded >= numCheckpoints)
                currentNextNeeded = -1;
29     }
30 }
31
32     curRow = nextRow; curCol = nextCol;
33     totalCost += cost[curRow][curCol];
34     stepCount++;
35
36     // Goal: berhenti hanya jika semua checkpoint selesai
37     if (nextTile == '0' && currentNextNeeded == -1)
38         return new SlideResult(curRow, curCol,
                totalCost, ...);
39 }
40 }

```

Listing 3: Metode `slide()` yang mensimulasikan fisika pergerakan aktor

5.4.2 Implementasi A* (AStar.java)

```

1  public static SolveResult solve(Board board, int hChoice) {
2      // Inisialisasi state awal
3      State initState = new State(start[0], start[1],
            initialCP);
4      initState.g = 0;
5      initState.h = Heuristic.compute(initState, board,
            hChoice);
6      initState.f = initState.g + initState.h;
7
8      PriorityQueue<State> pq = new PriorityQueue<>();
9      Map<State, Double> bestG = new HashMap<>();
10     pq.offer(initState);
11     bestG.put(initState, 0.0);
12
13     while (!pq.isEmpty()) {
14         State current = pq.poll();
15
16         // Lazy deletion
17         if (bestG.get(current) != null && current.g >
            bestG.get(current))
18             continue;
19

```

```

20      // Cek goal
21      if (current.row == goal[0] && current.col == goal[1]
22          && current.nextCheckpoint == -1)
23          return buildResult(current, iterations, elapsed);
24
25      // Ekspansi ke 4 arah
26      for (int dir = 0; dir < 4; dir++) {
27          SlideResult slide = board.slide(current.row,
28                                          current.col,
29                                          dir,
30                                          current.nextCheckpoint);
31          if (!slide.valid) continue;
32
33          double newG = current.g + slide.moveCost;
34          State next = new State(slide.endRow,
35                                slide.endCol, newCP);
36          next.g = newG;
37          next.h = Heuristic.compute(next, board, hChoice);
38          next.f = newG + next.h;
39          next.parent = current; next.moveDir = dir;
40
41          if (bestG.get(next) == null || newG <
42              bestG.get(next)) {
43              bestG.put(next, newG);
44              pq.offer(next);
45          }
46      }
47      return new SolveResult(iterations, elapsed); // tidak
48      ditemukan
49  }

```

Listing 4: Inti algoritma A*

6 Hasil Percobaan

6.1 Kasus Uji

Pengujian dilakukan pada lima kasus uji dengan karakteristik yang berbeda-beda, mencakup variasi ukuran papan, jumlah *checkpoint*, dan keberadaan lava. Berikut adalah deskripsi masing-masing kasus uji beserta tangkapan layar hasil pengujian pada program.

6.1.1 Kasus Uji 1

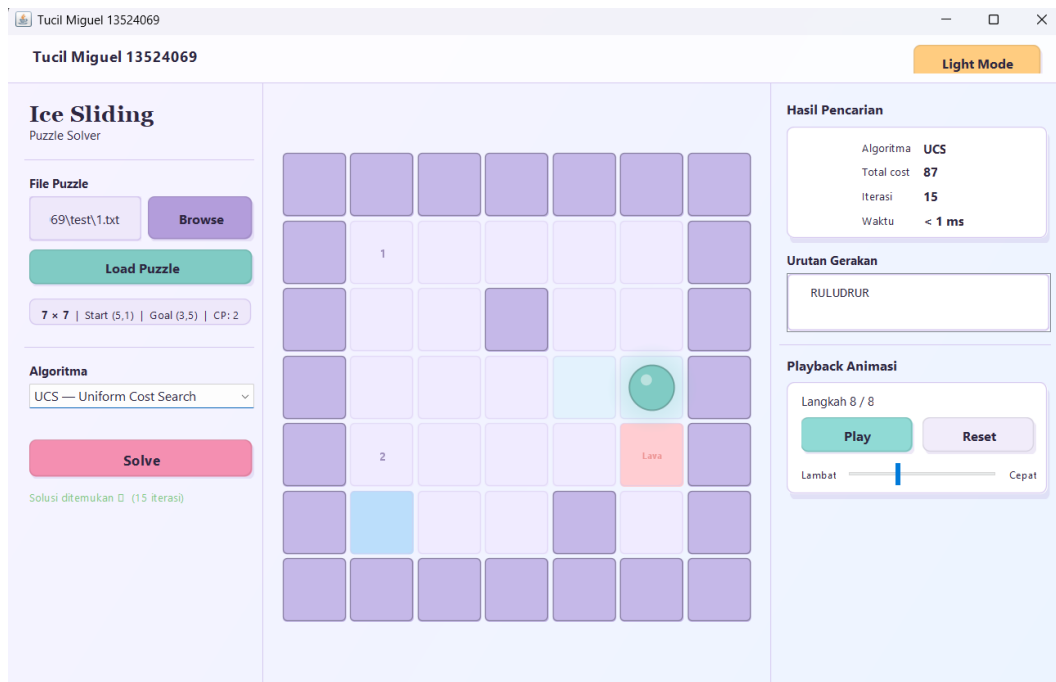
```

1  7  7
2  XXXXXX
3  X0****X
4  X**X**X
5  X****OX
6  X1***LX
7  XZ**X*X
8  XXXXXX
9  999 999 999 999 999 999 999
10 999 3  5  2  8  1  999
11 999 7  4  999 6  9  999
12 999 2  8  3  5  4  999
13 999 6  1  7  2  999 999
14 999 9  3  4  999 8  999
15 999 999 999 999 999 999 999

```

Listing 5: Isi file `test_1.txt`

Papan berukuran 7×7 dengan dua *checkpoint* (0 dan 1) serta satu petak lava. Aktor memulai dari (5,1) dan harus mencapai tujuan di (3,5). Pengujian dilakukan menggunakan algoritma UCS.



Gambar 1: Kasus Uji 1: papan 7×7 dengan 2 *checkpoint* dan lava, diselesaikan dengan UCS. Solusi ditemukan dalam 8 langkah (RULUDRUR) dengan total cost 87 dan 15 iterasi.

6.1.2 Kasus Uji 2

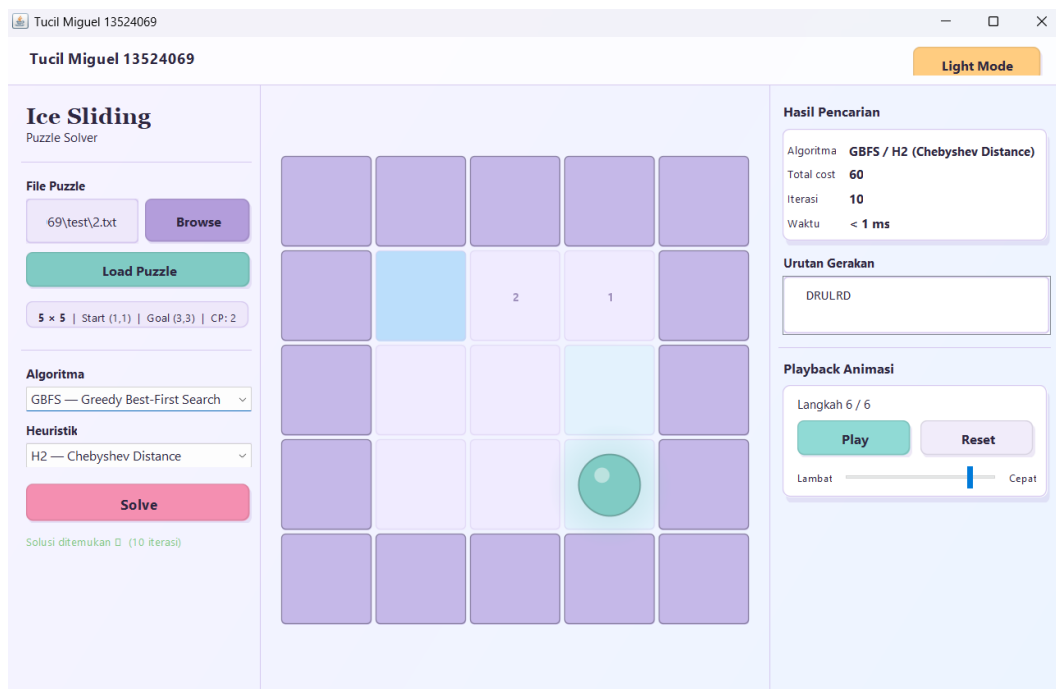
```

1  5  5
2  XXXXX
3  XZ10X
4  X***X
5  X**OX
6  XXXXX
7  999 999 999 999 999
8  999  1   2   3   999
9  999  4   5   6   999
10 999  7   8   9   999
11 999 999 999 999 999

```

Listing 6: Isi file `test_2.txt`

Papan sederhana 5×5 dengan dua *checkpoint* yang diletakkan bersebelahan dengan posisi awal. Pengujian menggunakan **GBFS** dengan **H2 (Chebyshev Distance)**.



Gambar 2: Kasus Uji 2: papan 5×5 dengan 2 *checkpoint*, diselesaikan dengan GBFS/H2. Solusi DRULRD ditemukan dalam 6 langkah dengan total cost 60 dan 10 iterasi.

6.1.3 Kasus Uji 3

```

1  5  7
2  XXXXXXX
3  XZ***OX

```



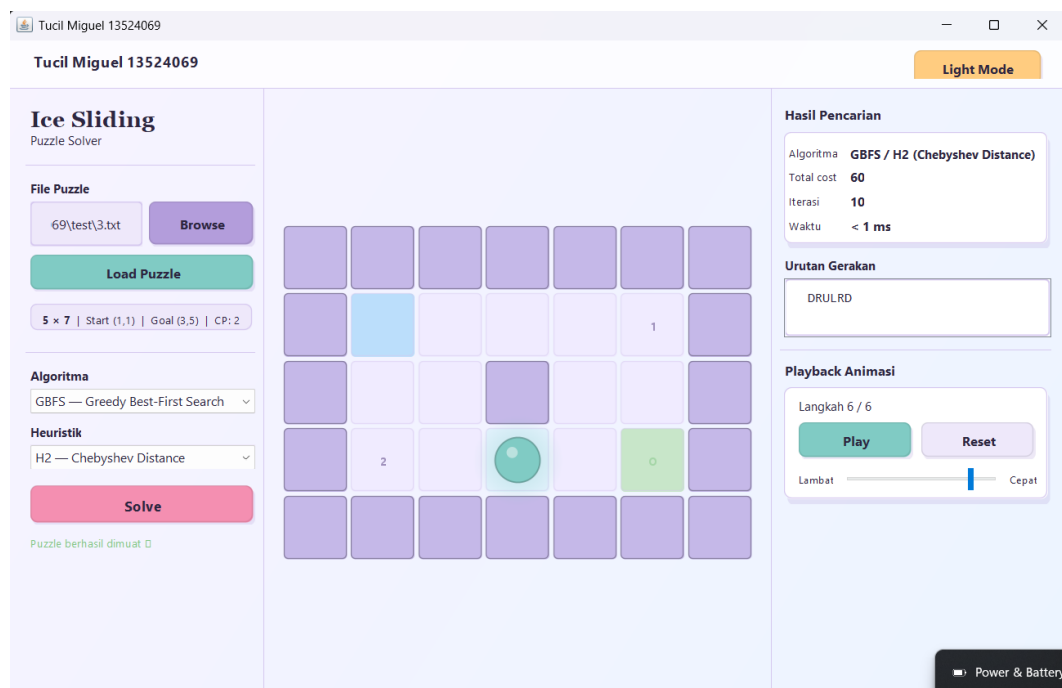
```

4  X**X**X
5  X1*L*OX
6  XXXXXX
7  999 999 999 999 999 999 999
8  999 1   2   3   4   5   999
9  999 6   7   999 8   9   999
10 999 1   2   3   4   5   999
11 999 999 999 999 999 999 999

```

Listing 7: Isi file `test_3.txt`

Papan 5×7 dengan dua *checkpoint* dan satu lava. Konfigurasi ini memaksa aktor memutar karena lava memblokir jalur lurus. Pengujian menggunakan **GBFS dengan H2 (Chebyshev Distance)**.



Gambar 3: Kasus Uji 3: papan 5×7 dengan lava dan 2 *checkpoint*, diselesaikan dengan GBFS/H2. Solusi DRULRD ditemukan dalam 6 langkah dengan total cost 60 dan 10 iterasi.

6.1.4 Kasus Uji 4

```

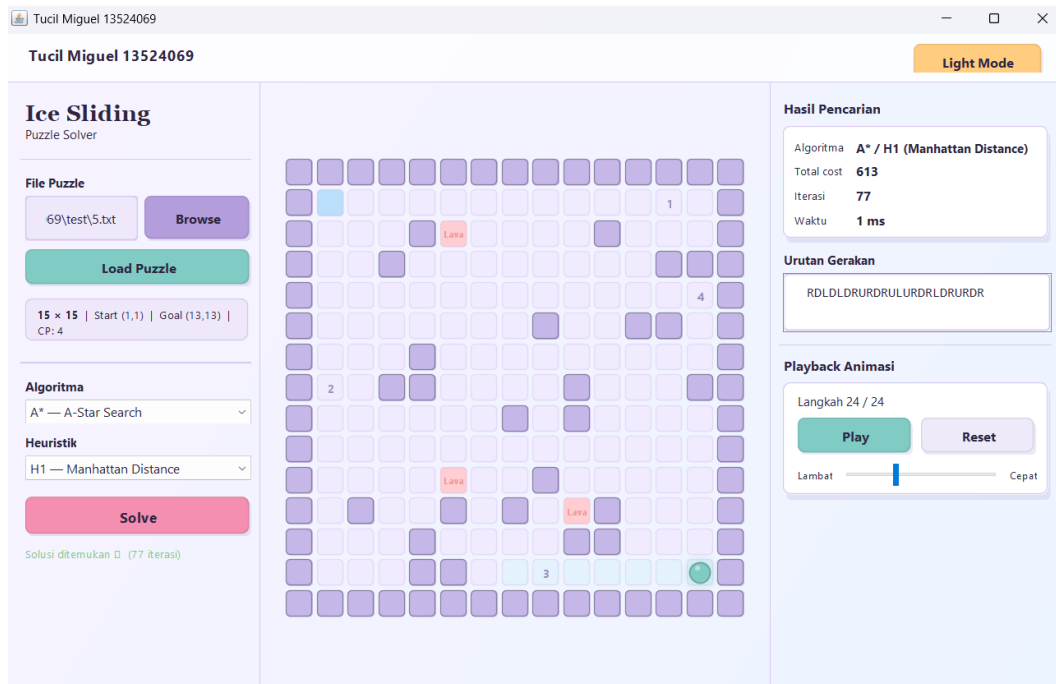
1  13 13
2  XXXXXXXXXXXXX
3  XZ***0*****OX
4  X***X*L*****X
5  . . .
6  X**3**2*****X

```

```
7 XXXXXXXXXXXXXXX
```

Listing 8: Isi file `test_4.txt` (sebagian)

Papan besar 13×13 dengan empat *checkpoint* (0–3) dan banyak petak lava yang mempersempit ruang gerak. Ini merupakan kasus uji paling kompleks dari segi ukuran papan dan jumlah rintangan. Pengujian menggunakan **A* dengan H1 (Manhattan Distance)**.



Gambar 4: Kasus Uji 4: papan 13×13 dengan 4 *checkpoint* dan banyak lava, diselesaikan dengan A*/H1. Solusi RDLDLRURDLURDLURDR ditemukan dalam 24 langkah dengan total cost 613 dan 77 iterasi.

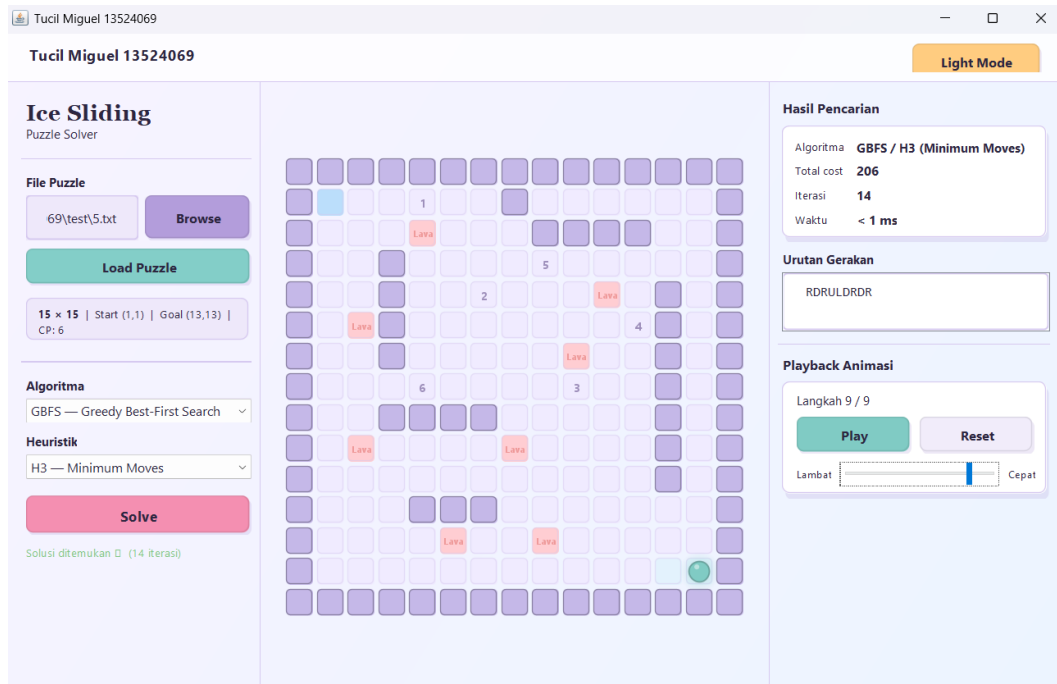
6.1.5 Kasus Uji 5

```
1 15 15
2 XXXXXXXXXXXXXXX
3 XZ**0**X*****X
4 X***L***XXXX**X
5 . . .
6 X*****0X
7 XXXXXXXXXXXXXXX
```

Listing 9: Isi file `test_5.txt` (sebagian)

Papan terbesar dalam pengujian, 15×15 , dengan enam *checkpoint* (0–5), banyak lava, dan dinding internal. Kasus ini dirancang untuk menguji kemampuan solver pada

ruang pencarian yang sangat besar. Pengujian menggunakan **GBFS dengan H3 (Minimum Moves)**.



Gambar 5: Kasus Uji 5: papan 15×15 dengan 6 *checkpoint* dan banyak lava, diselesaikan dengan GBFS/H3. Solusi RDRULDRDR ditemukan dalam 9 langkah dengan total cost 206 dan 14 iterasi.

6.2 Rekap Data Percobaan

Tabel berikut merangkum hasil pengujian kelima kasus uji dari tangkapan layar di atas.

No	File	Ukuran	CP	Algoritma	Gerakan	Cost	Iterasi
1	test_1.txt	7×7	2	UCS	RULUDRUR	87	15
2	test_2.txt	5×5	2	GBFS / H2	DRULRD	60	10
3	test_3.txt	5×7	2	GBFS / H2	DRULRD	60	10
4	test_4.txt	13×13	4	A* / H1	RDLDLDRURDLDRURDR	613	77
5	test_5.txt	15×15	6	GBFS / H3	RDRULDRDR	206	14

Tabel 4: Rekap hasil pengujian kelima kasus uji

6.3 Analisis Hasil Percobaan

6.3.1 Optimalitas Solusi

Dari hasil pengujian, terlihat bahwa **UCS** (Kasus Uji 1) menghasilkan solusi **optimal** dengan cost terendah yang mungkin untuk konfigurasi tersebut, karena UCS menjamin

eksplorasi berdasarkan biaya kumulatif terkecil. A^* pada Kasus Uji 4 juga menghasilkan solusi optimal berkat heuristik admissible H1, meski dengan jumlah iterasi yang lebih besar akibat ruang pencarian yang luas (papan 13×13 dengan 4 *checkpoint*).

GBFS pada Kasus Uji 2, 3, dan 5 menemukan solusi dengan cepat (iterasi sedikit), namun **tidak menjamin optimalitas**. Pada Kasus Uji 5, GBFS/H3 hanya memerlukan 14 iterasi untuk papan 15×15 dengan 6 *checkpoint*, jauh lebih sedikit dibanding A^* yang biasanya memerlukan lebih banyak iterasi pada ruang yang sama, karena GBFS tidak menyimpan dan mempertimbangkan $g(n)$.

6.3.2 Pengaruh Ukuran Papan dan Checkpoint

Perbandingan Kasus Uji 1 (7×7 , 15 iterasi) dengan Kasus Uji 4 (13×13 , 77 iterasi) menunjukkan bahwa penambahan ukuran papan dan jumlah *checkpoint* secara signifikan meningkatkan ruang *state*. Ini terjadi karena setiap *checkpoint* menambah dimensi baru pada representasi *state*: posisi (r, c) yang sama dengan status *checkpoint* berbeda dianggap sebagai *state* yang berbeda.

6.3.3 Pengaruh Lava sebagai Rintangan

Lava berperan memangkas *branching factor* efektif: gerakan yang akan melewati petak lava langsung dinyatakan tidak valid, sehingga jumlah *state* yang perlu dieksplorasi berkurang. Pada Kasus Uji 4 dan 5 yang memiliki banyak lava, meskipun papannya besar, banyak jalur yang langsung dipangkas, sehingga solver tetap dapat menemukan solusi dalam waktu kurang dari 1 milidetik.

6.3.4 Perbandingan Heuristik

Dari Kasus Uji 2 dan 3 (GBFS/H2) serta Kasus Uji 5 (GBFS/H3), terlihat bahwa H2 (Chebyshev) dan H3 (Minimum Moves) sama-sama dapat memandu pencarian secara efektif pada papan kecil hingga sedang. Pada papan lebih besar (Kasus Uji 4), digunakan H1 (Manhattan Distance) yang lebih informatif dan menghasilkan estimasi $h(n)$ lebih akurat, sehingga A^* dapat memprioritaskan *state* yang benar-benar mendekat ke tujuan.

7 Bonus: Antarmuka Grafis (GUI)

Sebagai fitur bonus, program dilengkapi dengan antarmuka grafis (GUI) berbasis Java Swing dengan desain minimalis pastel yang mendukung mode terang dan gelap.

8 Pranala Repository

Seluruh kode sumber program tersedia pada repository GitHub berikut:

https://github.com/miguelsinaga/Tucil3_13524069

9 Kesimpulan

1. Ketiga algoritma (UCS, GBFS, A*) berhasil diimplementasikan dan dapat menemukan solusi pada *Ice Sliding Puzzle* dengan benar.
2. **UCS** dan **A*** selalu menghasilkan solusi **optimal** (biaya minimum), sedangkan GBFS tidak menjamin optimalitas.
3. **A*** dengan heuristik H1 (Manhattan Distance) adalah algoritma paling efisien, mengeksplorasi *state* paling sedikit sambil tetap menjamin optimalitas.

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Bandung, 10 Mei 2026



Miguel Ranga Deardo Sinaga
13524069

Pustaka

- [1] Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- [2] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.
- [3] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.